

Image Restoration with Universal, Hard-Routed and Soft Mixture-of-Experts Autoencoders and Style-Conditioned Face-to-Sketch Generation with a Conditional GAN

Iman Suleman

Roll No. 23i-0618, Section CS-G

Department of Computer Science, FAST NUCES Islamabad

Generative AI, Assignment 1

Abstract—This report covers four related models I built for the Generative AI course. The first three restore Oxford-IIIT Pet images damaged by salt-and-pepper noise, Gaussian blur or black rectangles: one universal denoising autoencoder, a classifier that sends each image to one of three specialist autoencoders, and a soft mixture-of-experts whose gate starts from that classifier. The fourth is a conditional GAN that turns a face photo into a sketch in one of three FS2K styles. All models were tuned with Optuna, tracked with MLflow and exported to ONNX, where the outputs match PyTorch to within 2.1×10^{-6} . They run together in one React and FastAPI app started with Docker Compose. On the fixed test set the soft mixture-of-experts is best on single corruptions, with a mean PSNR of 24.22 dB against 23.63 dB for the universal autoencoder and 23.33 dB for hard routing. When two corruptions are applied at once the universal autoencoder wins (19.42 dB against 19.07 and 18.88 dB). The classifier is 99.9% accurate. The cGAN reaches PSNR 15.61 dB and SSIM 0.538, compared with 6.73 dB and 0.289 for using the grayscale photo as the sketch.

Index Terms—denoising autoencoder, mixture of experts, image restoration, conditional GAN, Optuna, ONNX

I. INTRODUCTION

Real photos rarely come with a label saying what is wrong with them. A restoration system can deal with this in three ways: train one model for every kind of damage, detect the damage and call a specialist, or blend several specialists with learned weights. Tasks 1 to 3 compare these three options under the same conditions: the same data split, the same runtime corruption code, the same fixed test set and the same metrics. Task 4 is a different problem. It translates a photo into a sketch, and a style label has to control the output.

The report contains a corruption pipeline that can be reproduced, with fixed validation and test sets (Section IV); a comparison of universal, hard-routed and soft-routed restoration, including oracle versus predicted routing and a test with two corruptions at once (Sections V and VIII); a pix2pix-style GAN where the style enters both the generator and the discriminator; Optuna studies for every model; and a containerised app for all four tasks (Section IX). The code and the demo video are listed in the next section.

II. PROJECT LINKS

- **Code repository (GitHub):** <https://github.com/ImaanSuleiman/genai-assignment1>
- **Demo video (YouTube):** <https://youtu.be/KFsUBjwYsj8>

III. RELATED RESEARCH AND DESIGN DECISIONS

Denoising autoencoders. Vincent et al. showed that training an autoencoder to rebuild a clean image from a damaged one makes it learn structure instead of copying [1]. DnCNN [2] and context encoders [3] showed that convolutional networks cope with noise and missing regions, and all-in-one restoration work [4] raises the question I study here: can one network for several kinds of damage compete with specialists? I kept a plain encoder, bottleneck, decoder shape with no unrestricted skip connections, as the assignment requires. A full skip would let the decoder copy the noisy input and defeat the bottleneck. One narrow skip is available as an option (`--limited-skip`) so its effect can be measured later.

Loss functions. L1 gives sharper restorations than L2, and the structural similarity index (SSIM) [5] rewards local contrast and structure. I train with $\alpha \mathcal{L}_{L1} + (1 - \alpha)(1 - \text{SSIM})$ and let Optuna pick α . The validation objective $L1 + (1 - \text{SSIM})$ does not use α . Otherwise the search would simply prefer whichever weight makes the objective numerically smallest.

Mixtures of experts. Adaptive mixtures of local experts [6] learn a gate that weights the experts' outputs, and sparsely gated layers [7] scale the idea up. Both report that one expert can end up receiving nearly everything unless the load is balanced. To avoid this in Task 3 I start the gate and experts from Task 2, warm up the gate with the experts frozen, use balanced batches and add the balance term $\sum_k (\bar{w}_k - \frac{1}{4})^2$. Optuna trials are pruned when routing collapses, meaning any mean weight above 0.7 or below 0.03.

Conditional GANs. The cGAN [8] feeds side information to both networks. Pix2pix [9] adds a U-Net [10] generator, a PatchGAN discriminator and an L1 term (weight 100) for paired translation. I follow that recipe at 128×128 with six down-sampling stages, so the bottleneck is 2×2 . The style label is a learned embedding that goes in three places: it is

TABLE I
MAIN DESIGN DECISIONS AND THE EVIDENCE BEHIND THEM.

Decision	Alternatives considered	Reason / evidence
No unrestricted skips in UDAE	U-Net style skips	Skips copy corrupted input [1]; ablation switch provided
L1 + SSIM loss	L2, L1 only	L1 sharper than L2; SSIM preserves structure [5]
Spatial 8×8 latent	flat dense vector	Keeps spatial layout; dense layers over 128^2 maps add parameters without benefit
Avg+max pooling classifier	avg pooling only	Max pooling keeps local evidence (isolated noise, mask edges)
Balanced batches	random class mix	Needed for unbiased classifier and for $\bar{w}_k$ in the balance loss
Gate init. from Task 2	random gate	Avoids early expert collapse [7]
FiLM + input-channel style	label only in the UI	Assignment requires style inside G and D
Instance norm in GAN	batch norm	Small GAN batches make batch statistics unreliable [12]

added to the generator input as extra channels, it is injected into every decoder block with FiLM scale and shift [11], and it is added to the discriminator input. That makes it part of the model and not just a label in the interface. I use instance normalisation [12] instead of batch normalisation because the batch sizes in the Optuna search are small.

Table I lists the main design decisions and the evidence behind each.

Hyper-parameter search. Every model uses Optuna [13] with the TPE sampler and median pruning. Studies are saved in SQLite so they can be resumed and inspected.

IV. DATASETS AND PREPROCESSING

A. Oxford-IIIT Pet (Tasks 1–3)

The Oxford-IIIT Pet dataset [14] has cat and dog photos from 37 breeds. I convert the official `trainval` images to RGB, resize them to 128×128 and split them 80/20 into training and validation sets with seed 42 (2944 training, 736 validation images). The official test images are not touched until the final evaluation. All three tasks share this split, and breed labels are never used.

B. Runtime corruption pipeline

Nothing corrupted is stored on disk. Every time a training image is loaded, one of four conditions is drawn with equal probability and a severity is sampled (Table II). The condition also serves as the label for the corruption classifier. Salt-and-pepper noise sets a pixel to black or white with equal chance. Blur is an OpenCV Gaussian blur with a kernel of 3, 5 or 7. Occlusion draws one to three black rectangles whose combined area covers 10 to 35% of the image, at random positions.

Validation and test corruptions are fixed so results can be compared between runs. The validation manifest holds one

TABLE II
CORRUPTION CONFIGURATION.

Condition	Training (sampled per load)	Test (fixed low/medium/high)
Clean	original image	original image
Salt-and-pepper	$p \sim U(0.02, 0.15)$	$p \in \{0.03, 0.08, 0.15\}$
Gaussian blur	$k \in \{3, 5, 7\}, \sigma \sim U(0.5, 2.5)$	$(k, \sigma) \in \{(3, 0.7), (5, 1.5), (7, 2.5)\}$
Occlusion	1–3 rectangles, joint area 10–35%	1/2/3 rectangles, area $\approx 10/20/35\%$

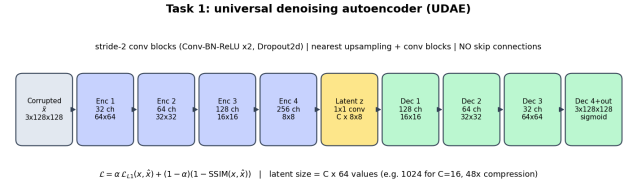


Fig. 1. Architecture of the universal denoising autoencoder.

balanced corruption per validation image. The test manifest holds the clean image plus three corruptions at three severities, ten entries per image. Both are generated once and saved as JSON, with the type, severity, mask coordinates, blur settings and random seed of every entry. For reporting, validation severities are grouped into low, medium and high using the tertiles of the training range.

C. FS2K (Task 4)

FS2K [15] has 2,104 paired photos and sketches in three styles. I use the official train and test split and hold out 15% of the training part for validation, stratified by style with seed 42. Photos and sketches are resized to 128×128 . Augmentation (a horizontal flip and a random crop resized back) draws one random transformation and applies it to both images of a pair, so the pairing stays correct.

V. METHODOLOGY

A. Task 1: universal denoising autoencoder

The UDAE (Fig. 1) has four stride-2 encoder stages, a 1×1 convolution down to a latent tensor of $C \times 8 \times 8$ values, and a decoder made of nearest-neighbour up-sampling and convolution blocks that ends in a sigmoid. With $C = 16$ the latent has 1,024 numbers, 48 times fewer than the $3 \times 128 \times 128$ input, so the network cannot simply copy its input. It is trained with

$$\mathcal{L}_{UDAE} = \alpha \mathcal{L}_{L1}(x, \hat{x}) + (1 - \alpha)(1 - \text{SSIM}(x, \hat{x})), \quad (1)$$

with $\hat{x} = D(E(\tilde{x}))$. The network is never told which corruption was applied.

B. Task 2: classifier and hard-routed specialists

The classifier is a convolutional network whose channel widths were chosen by Optuna, with average and max global

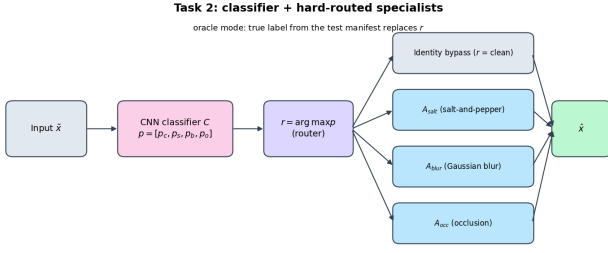


Fig. 2. Hard-routed restoration pipeline.

pooling. It is trained with cross-entropy on batches that hold exactly one quarter of each class. The three specialists (salt-and-pepper, blur, occlusion) share one architecture from a shared Optuna search, but they are trained separately, with different seeds and weights, each only on its own corruption and with the clean image as the target. At inference (Fig. 2 shows the pipeline)

$$\hat{x} = \begin{cases} \tilde{x}, & r = \text{clean} \\ A_r(\tilde{x}), & \text{otherwise} \end{cases}, \quad r = \arg \max_k p_k, \quad (2)$$

so clean images take an identity bypass. I evaluate two modes. Oracle routing uses the true label from the manifest and isolates the quality of the specialists. Predicted routing uses the classifier and measures the system as it would be deployed.

C. Task 3: jointly trained soft mixture-of-experts

The gate G is the Task 2 classifier, and the experts are the three specialists plus an identity branch. With $w = \text{softmax}(G(\tilde{x})/\tau)$ the output is $\hat{x} = w_0\tilde{x} + w_1A_{\text{salt}}(\tilde{x}) + w_2A_{\text{blur}}(\tilde{x}) + w_3A_{\text{occ}}(\tilde{x})$, which is differentiable in both the gate and the experts (Fig. 3). Stage A trains only the gate with the experts frozen (learning rate 3η). Stage B unfreezes everything and fine-tunes at learning rate η . The loss is

$$\mathcal{L}_{MoE} = \lambda_1 \mathcal{L}_{L1} + \lambda_s (1 - \text{SSIM}) + \lambda_c \mathcal{L}_{CE} + \lambda_b \sum_{k=0}^3 \left(\bar{w}_k - \frac{1}{4} \right)^2, \quad (3)$$

where $\lambda_s = 1 - \lambda_1$, the cross-entropy uses the raw gate logits and the runtime corruption label, and $\bar{w}_k$ is the mean weight of branch k in a class-balanced batch. I use the quadratic balance loss suggested in the assignment. It is smallest when the batch-mean weights are uniform, which is the right target because a balanced batch has 25% of each condition.

D. Task 4: style-conditioned cGAN

The generator is a six-stage U-Net with skip connections and style FiLM in every decoder block (Fig. 4). The discriminator is a PatchGAN over the photo, the sketch and the style embedding stacked together. The discriminator minimises $\frac{1}{2} [\text{BCE}(D(x, y, s), 1) + \text{BCE}(D(x, G(x, s), s), 0)]$ and the generator minimises $\mathcal{L}_G = \mathcal{L}_{adv} + \lambda_{L1} \|y - G(x, s)\|_1$. Five loss terms (D-real, D-fake, G-adversarial, G-L1 and validation) are logged every epoch, and the same four validation photos are rendered in all three styles at fixed intervals (Fig. 31).

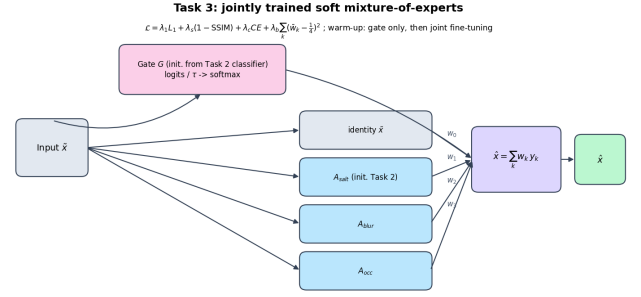


Fig. 3. Soft mixture-of-experts model.

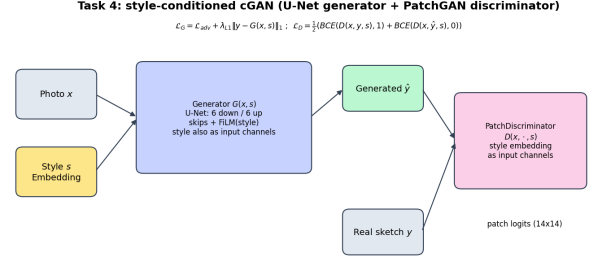


Fig. 4. Conditional GAN architecture.

VI. HYPER-PARAMETER OPTIMISATION

Every model is tuned with an Optuna TPE study and median pruning. Studies are saved in SQLite, every trial is a nested MLflow run, and trials use a shorter epoch budget. The chosen configuration is then retrained for the full schedule. Table III summarises the studies and Tables IV–VIII list the search spaces. The Task 1 objective is validation L1 + (1 - SSIM) on the fixed validation manifest. The classifier maximises validation macro-F1. The specialist search averages the objective over the three specialists, so the shared architecture suits all of them. The MoE search uses the same objective as Task 1 and prunes collapsed routing. The GAN search uses L1+(1-SSIM) between the generated and the paired sketch on the validation set, with fewer epochs. That is only a proxy, because it rewards fidelity and not realism (see Section X).

Table III and Figs. 5–9 show how each search went and which settings mattered. The history plots show how much tuning gained. In Task 1 the best objective improved from 0.451 at the first trial to 0.412 at trial 12 and did not improve afterwards. The classifier barely moved (Fig. 6): every finished trial scored a macro-F1 between 0.971 and 0.989, so tuning mattered little for that easy task. The specialist search gained the most, from 0.585 to 0.439 (Fig. 7). The MoE search (Fig. 8) went from 0.210 to 0.191 and its best trial was one of the last, so with only 20 trials it probably had not converged. The GAN objective fell from 0.627 to 0.563. The importance plots rank the parameters. In Task 1 the loss weight α mattered most (importance 0.47), then the learning rate (0.18) and the base width and latent size (0.12 each). The chosen values were

TABLE III
OPTUNA STUDIES: TRIAL COUNTS, BEST OBJECTIVE VALUE AND THE FINAL SELECTED CONFIGURATION.

Study	Trials	Done	Pruned	Best obj.	Selected configuration
Task 1 UDAE	25	12	13	0.4119	lr=0.000262, batch_size=16, latent_ch=32, base=48, dropout=0.0206, alpha=0.532
Task 2 classifier	20	14	6	0.9891	lr=0.00271, batch_size=32, channels=small, dropout=0.0226, weight_decay=2e-05
Task 2 specialists	15	13	2	0.4388	lr=0.000874, latent_ch=64, base=48, batch_size=16, alpha=0.697
Task 3 soft MoE	20	12	8	0.1907	lr=0.000528, tau=1.37, lam_ce=0.0274, lam_bal=0.105, lam_l1=0.649
Task 4 cGAN	15	10	5	0.5626	lr_g=0.00036, lr_d=0.000304, batch_size=8, base=64, dropout=0.0969, style_dim=16, lam_l1=197

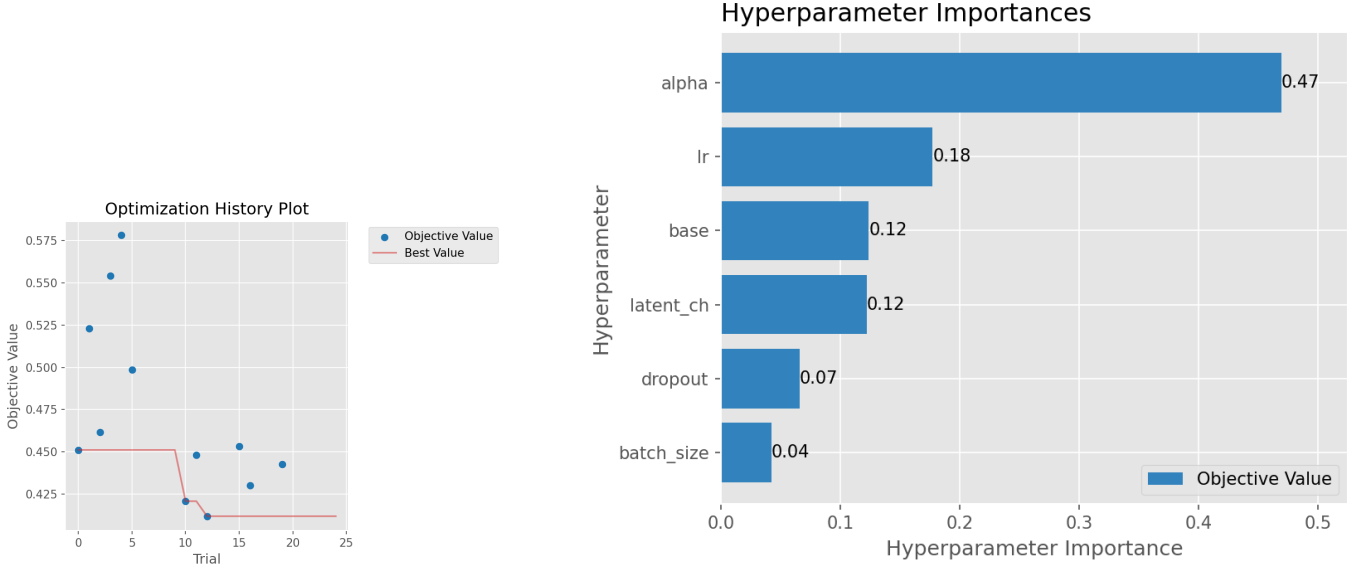


Fig. 5. Task 1 (UDAE) Optuna study: optimisation history (left) and hyper-parameter importance (right).

TABLE IV
OPTUNA SEARCH SPACE FOR TASK 1 UDAE (25 TRIALS, 6 EPOCHS PER TRIAL; OBJECTIVE: MIN VAL(L1 + (1 - SSIM)) ON VAL MANIFEST).

Hyper-parameter	Search space
lr	loguniform[1e-4, 3e-3]
batch_size	categorical[16, 32, 64, 128]
latent_ch	categorical[8, 16, 32, 64] (latent = latent_ch x 8 x 8)
base	categorical[16, 32, 48] (encoder channels base, 2*base, 4*base, 8*base)
dropout	uniform[0.0, 0.3]
alpha	uniform[0.5, 0.95] (L1 weight; SSIM weight = 1 - alpha)

$\alpha = 0.53$, latent $32 \times 8 \times 8$ and learning rate 2.6×10^{-4} . An α near 0.5 makes sense because the validation objective weighs L1 and $1 - \text{SSIM}$ equally, so the training loss that matches it works best. The 32-channel latent (2,048 values, 24 times smaller than the input) won over smaller ones, but latent size had low importance, so the search showed no strong preference. Only 12 of the 25 trials finished (13 were pruned), so these importance values are noisy.

For the classifier, dropout (0.54), learning rate (0.21) and

TABLE V
OPTUNA SEARCH SPACE FOR TASK 2 CLASSIFIER (20 TRIALS, 5 EPOCHS PER TRIAL; OBJECTIVE: MAX VAL MACRO-F1).

Hyper-parameter	Search space
lr	loguniform[1e-4, 3e-3]
batch_size	categorical[16, 32, 64, 128] (balanced: batch/4 per class)
channels	categorical[small(16-128), medium(32-256), large(32-256x2)]
dropout	uniform[0.0, 0.5]
weight_decay	loguniform[1e-6, 1e-2]

weight decay (0.20) mattered most, and channel width hardly mattered (0.03). That fits an easy task: almost every finished trial passed 97% validation accuracy, and the smallest network won. For the specialists, α (0.72) and the learning rate (0.25) dominated, and the chosen $\alpha = 0.70$ favours L1 more than Task 1 did. For the MoE, the gate temperature (0.39), the balance weight (0.32) and the learning rate (0.22) mattered most, and a softer gate ($\tau = 1.37$) was preferred. The cross-entropy weight mattered little (0.027).

For the GAN, the two learning rates (0.30 and 0.29), dropout

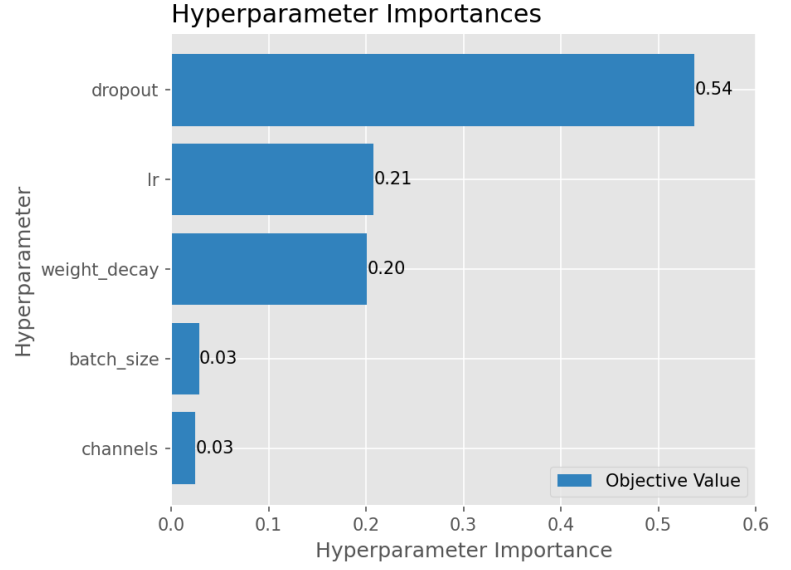
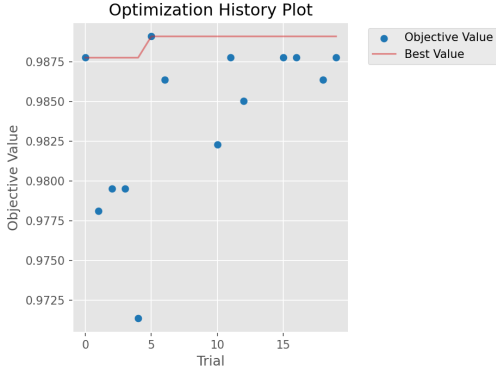


Fig. 6. Task 2 classifier study: optimisation history (left) and importance (right).

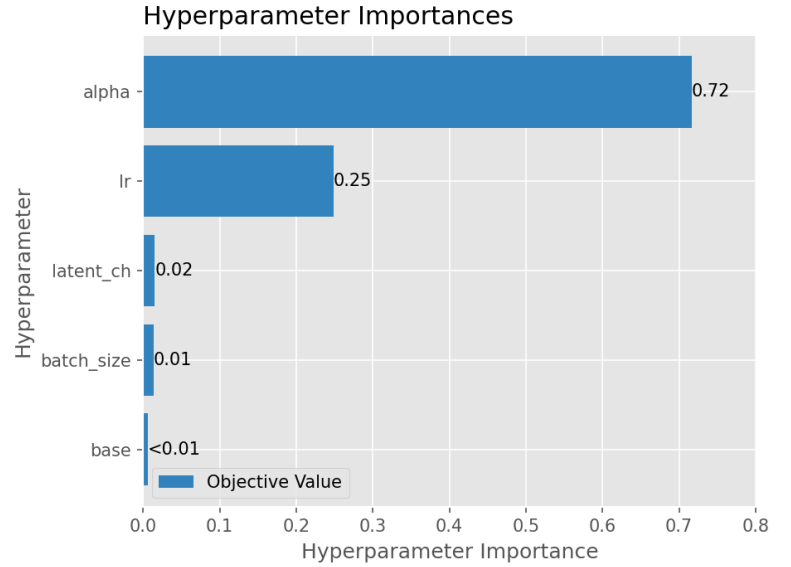
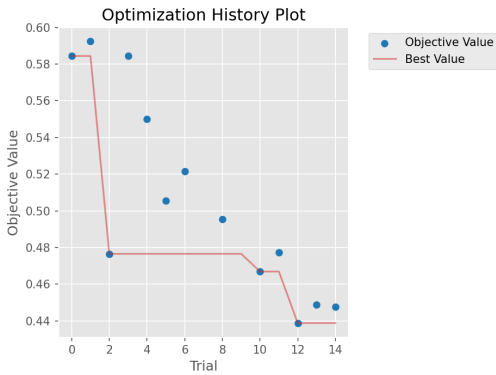


Fig. 7. Task 2 specialist study (shared architecture): history (left) and importance (right).

(0.24) and λ_{L1} (0.17) mattered, while style dimension, batch size and width had no measurable effect. The chosen $\lambda_{L1} \approx 197$ is about twice the usual pix2pix value of 100. Only 10 of the 15 GAN trials finished, each of 10 epochs, and the objective is a fidelity proxy, so read its ranking with caution.

VII. EXPERIMENTAL SETUP

A. Training procedure

All models are written in PyTorch and were trained on a Google Colab runtime with an NVIDIA T4 GPU. I use Adam (AdamW for the classifier) with cosine learning-rate decay, and Adam with $\beta_1 = 0.5$ for the GAN. All seeds are 42.

The tasks are trained in the order they depend on each other. First the UDAE (Task 1) trains for 60 epochs on images that are corrupted as they are loaded. Next the classifier trains for 40 epochs on class-balanced batches, and each of the three specialists trains for 40 epochs on its own corruption only. The soft MoE (Task 3) then starts from those weights and trains for 2 warm-up epochs (gate only) plus 30 joint epochs. The cGAN (Task 4) trains for 150 epochs with the discriminator and generator updated alternately. Before each full run, Optuna chose the hyper-parameters with shorter trials of 6 epochs (UDAE), 5 (classifier), 4 (specialists), 4 (MoE) and 10 (cGAN), and the chosen configuration was then retrained

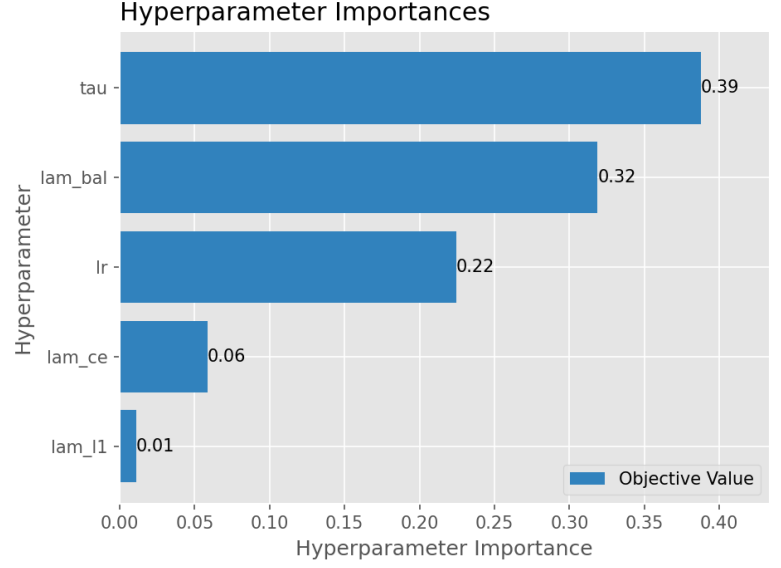
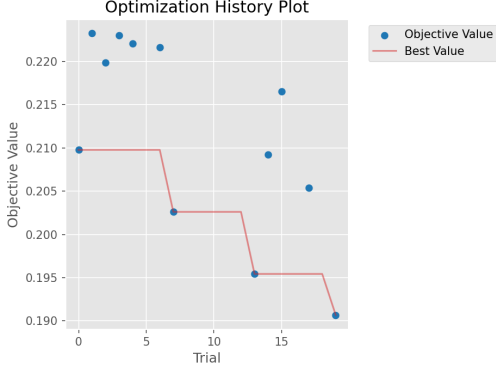


Fig. 8. Task 3 soft-MoE study: history (left) and importance (right).

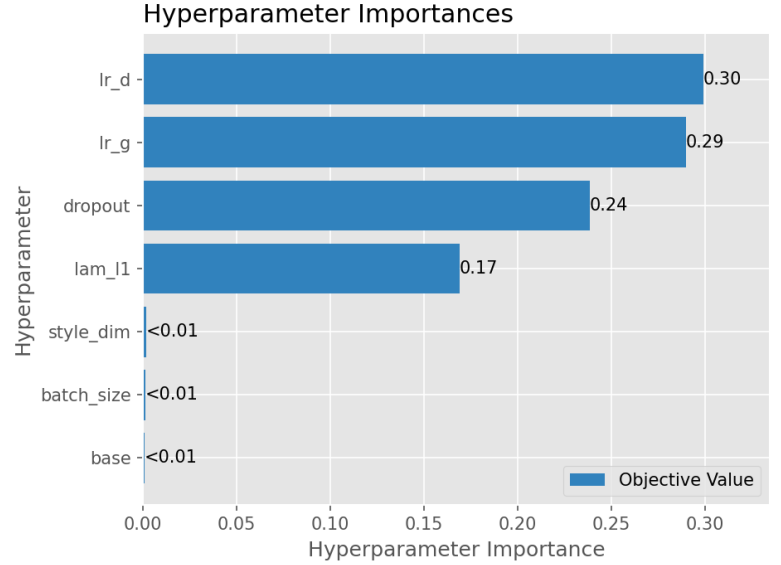
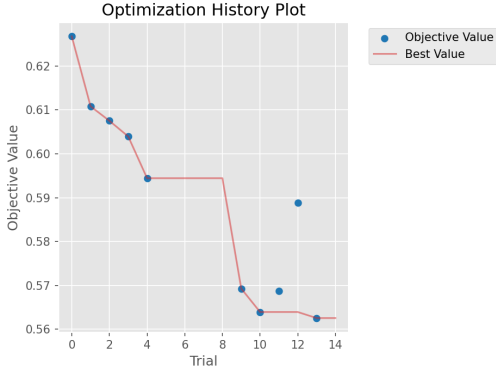


Fig. 9. Task 4 cGAN study: history (left) and importance (right).

for the full schedule.

B. Evaluation protocol

Metrics are PSNR, SSIM and L1 against the clean target on the fixed test manifest. Task 2 also reports accuracy, macro precision, recall and F1, and a normalised confusion matrix. Hyper-parameters, per-epoch losses, checkpoints and sample images are tracked in MLflow, and Fig. 10 shows screenshots of its UI. Figs. 10 and 11 show that every task has its own experiment, with one parent run per Optuna study, one nested run per trial and one long run for the final training.

VIII. RESULTS AND ANALYSIS

A. Task 1: universal autoencoder

Analysis. The UDAE removes salt-and-pepper noise well (PSNR 16.38 $\rightarrow$ 24.76 dB, SSIM 0.412 $\rightarrow$ 0.787) and recovers occlusions partly (13.61 $\rightarrow$ 21.30 dB). It does not help with Gaussian blur. The blurred input already scores 27.78 dB and the output scores 24.84 dB, and SSIM also falls (0.828 $\rightarrow$ 0.780). The twelve examples in Fig. 12 show the same thing in the error maps. For salt-and-pepper and blur the error is faint and spread over texture and edges, while for occlusion it is a bright block exactly where the black rectangles

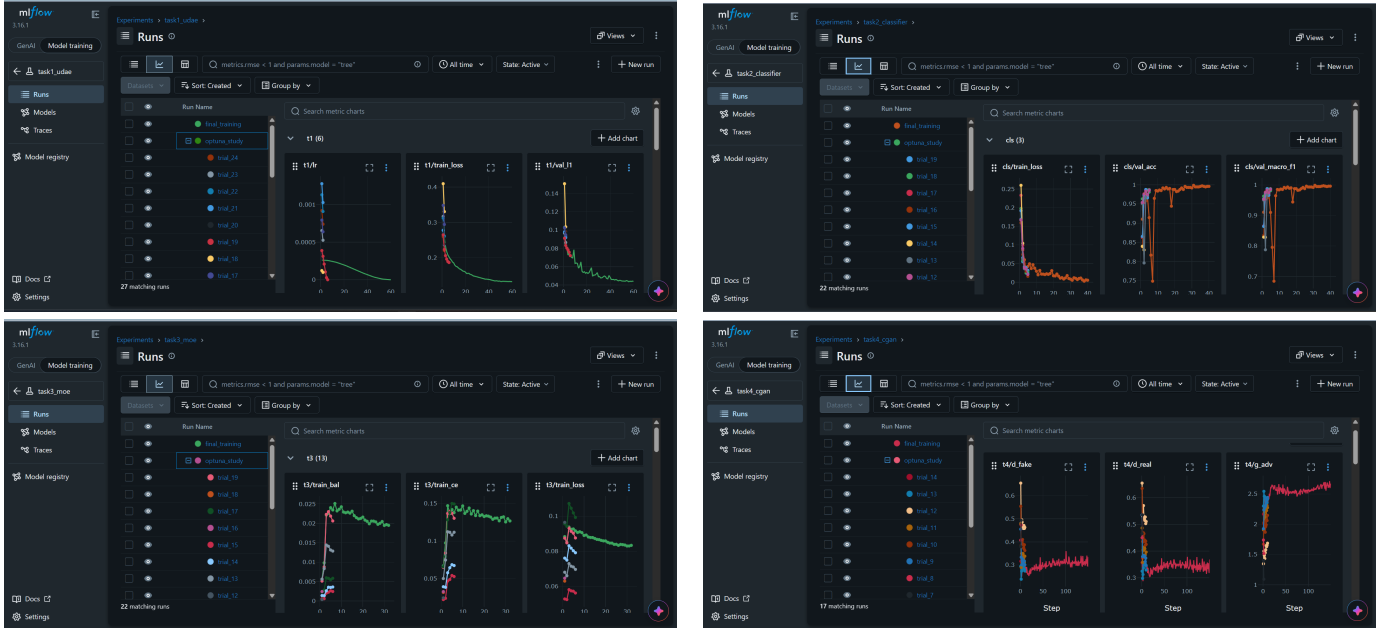


Fig. 10. MLflow experiment tracking for each task (top left to bottom right: Task 1 UDAE, Task 2 classifier, Task 3 soft MoE, Task 4 cGAN). Each experiment holds a parent Optuna study with one nested run per trial, plus the final training run whose curves (the long line in each chart) are logged per epoch.

TABLE VI

OPTUNA SEARCH SPACE FOR TASK 2 SPECIALISTS (15 TRIALS, 4 EPOCHS PER TRIAL; OBJECTIVE: MIN MEAN OVER 3 SPECIALISTS OF VAL(L1 + (1 - SSIM))).

Hyper-parameter	Search space
lr	loguniform[1e-4, 3e-3]
latent_ch	categorical[8, 16, 32, 64]
base	categorical[16, 32, 48]
batch_size	categorical[16, 32, 64, 128]
alpha	uniform[0.5, 0.95] (L1 weight; SSIM weight = 1 - alpha)

TABLE VII

OPTUNA SEARCH SPACE FOR TASK 3 SOFT MOE (20 TRIALS, 4 EPOCHS PER TRIAL; OBJECTIVE: MIN VAL(L1 + (1 - SSIM))); PRUNED ON ROUTING COLLAPSE (MAX MEAN WEIGHT ≥ 0.7 OR MIN ≤ 0.03)).

Hyper-parameter	Search space
lr	loguniform[1e-5, 1e-3] (fine-tuning LR; warm-up LR = 3 x lr)
tau	loguniform[0.3, 3.0]
lam_ce	loguniform[0.01, 1.0]
lam_bal	loguniform[1e-3, 1.0]
lam_l1	uniform[0.5, 0.95] (L1 weight; SSIM weight = 1 - lam_l1)

were, and the output there is a smooth guess. The training curves (Fig. 14) fall steadily and flatten after about 45 epochs, with the validation objective ending near 0.27. The training loss sits at about half that value only because it is the α -weighted loss and $\alpha \approx 0.5$, not because the model generalises better than it should. The two curves never diverge, so I see no sign of overfitting and no reason to train longer.

The clean row explains why. The autoencoder reproduces a

TABLE VIII

OPTUNA SEARCH SPACE FOR TASK 4 cGAN (15 TRIALS, 10 EPOCHS PER TRIAL; OBJECTIVE: MIN VAL(L1 + (1 - SSIM)) OF GENERATED VS PAIRED SKETCH).

Hyper-parameter	Search space
lr_g	loguniform[1e-4, 1e-3]
lr_d	loguniform[5e-5, 1e-3]
batch_size	categorical[4, 8, 16, 32]
base	categorical[32, 48, 64] (base channel count of G and D)
dropout	uniform[0.0, 0.5] (generator decoder dropout)
style_dim	categorical[8, 16, 32] (style-embedding dimension)
lam_l1	loguniform[10, 200] (L1 reconstruction weight, initial value 100)

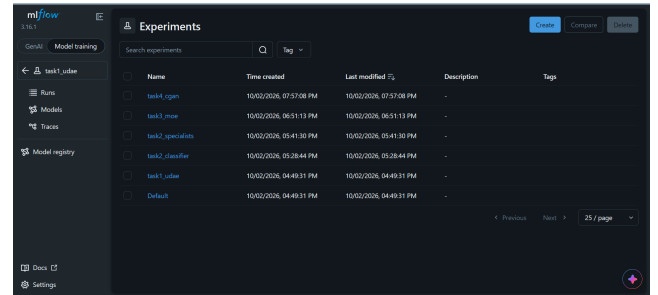


Fig. 11. MLflow experiment list: one experiment per training stage (Task 1, Task 2 classifier, Task 2 specialists, Task 3, Task 4).

perfectly clean image only to 24.94 dB, because the 8×8 bottleneck cannot store every detail. The outputs for salt-and-pepper (24.76) and blur (24.84) are within 0.2 dB of this ceiling, so the denoising itself is essentially solved and

Task 1: universal denoising autoencoder (12 test examples)

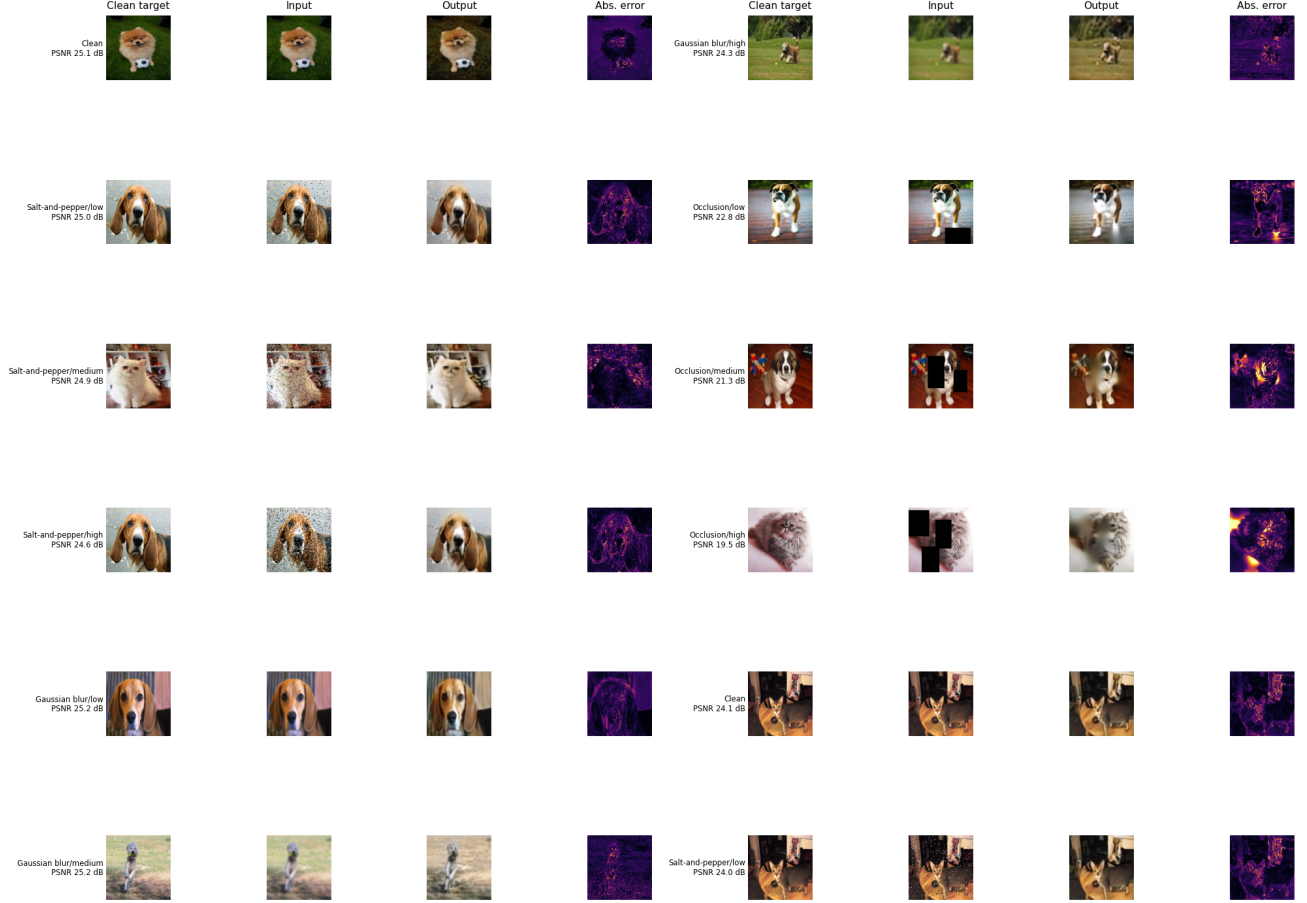


Fig. 12. Task 1: twelve test examples (clean target, corrupted input, reconstruction, absolute error).

TABLE IX
TASK 1 TEST RESULTS BY INPUT CONDITION (“IN” = CORRUPTED INPUT
VS. CLEAN TARGET, “OUT” = UDAE OUTPUT).

Condition	PSNR in	PSNR out	SSIM in	SSIM out	L1 out
Clean	100.00	24.94	1.000	0.800	0.0416
Salt-and-pepper	16.38	24.76	0.412	0.787	0.0425
Gaussian blur	27.78	24.84	0.828	0.780	0.0420
Occlusion	13.61	21.30	0.747	0.715	0.0570

what remains is the reconstruction loss of the architecture. Occlusion sits 3.6dB below the ceiling. That gap is real information loss: the pixels under the black rectangles are gone and the model can only paint a plausible guess. The error grows with the covered area (22.87, 21.41 and 19.60 dB at low, medium and high severity). This is the main weakness of a bottleneck autoencoder judged by PSNR: it makes clean and blurred images worse. (The 100dB shown for clean images with the identity bypass is a capped value.)

Fig. 13 shows the four lowest-PSNR cases, and all four are the same test image, a cat on a large black background, under occlusion and salt-and-pepper noise. The outputs have a

TABLE X
TASK 1 TEST RESULTS BY CORRUPTION AND FIXED SEVERITY LEVEL.

Corruption	Severity	PSNR in	PSNR out	SSIM in	SSIM out
Salt-and-pepper	low	20.13	24.92	0.626	0.797
Salt-and-pepper	medium	15.87	24.82	0.372	0.789
Salt-and-pepper	high	13.14	24.53	0.237	0.774
Gaussian blur	low	32.27	25.14	0.947	0.800
Gaussian blur	medium	26.73	25.08	0.820	0.790
Gaussian blur	high	24.33	24.31	0.716	0.751
Occlusion	low	16.70	22.87	0.880	0.761
Occlusion	medium	13.32	21.41	0.763	0.722
Occlusion	high	10.80	19.60	0.597	0.662

grey halo spreading from the cat into the background. So the failure comes from this image, since large flat dark regions are reproduced poorly, and not from the corruption type. I did not run an ablation to confirm the cause.

B. Task 2: hard routing

Analysis. The classifier is almost perfect: accuracy 0.9989 and macro-F1 0.998 over 36,690 test entries. The clean class is the least reliable (precision 0.993, recall 0.995), which is

Task 1: failure cases (lowest PSNR)

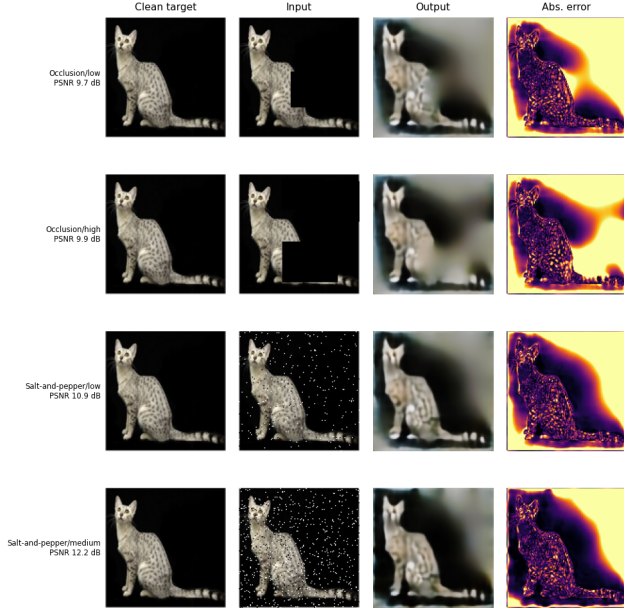


Fig. 13. Task 1 failure cases (lowest PSNR).

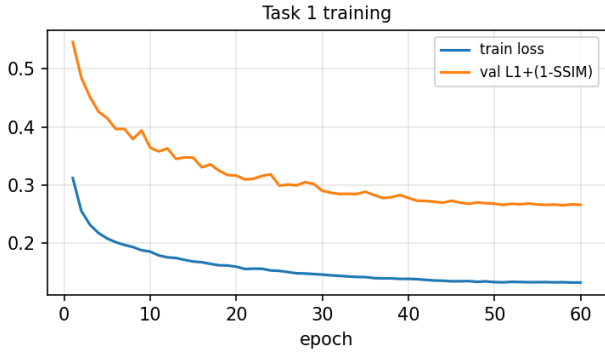


Fig. 14. Task 1 training and validation curves.

expected, since a clean image is hardest to tell apart from a low-severity corruption. Only 42 entries (0.1%) were misrouted, and the normalised confusion matrix (Fig. 15) rounds to the identity, with every diagonal cell at 0.99 or 1.00. The classifier curves (Fig. 17) show how easy the task is: validation accuracy is above 0.96 after two epochs and reaches about 0.99 after 20. There is one sharp dip to about 0.7 at epoch 7 that recovers by epoch 8; I did not investigate it, and it does not affect the final result. So oracle and predicted routing give the same PSNR on the corrupted classes to within 0.01 dB. The only visible cost is on clean images, where the mean drops from 100 to 99.66 dB because the few clean images sent to an expert come back at about 24 dB instead of being copied exactly.

All four examples in Fig. 16 are clean images sent to the blur or occlusion expert. The blur expert softens them (23–25 dB) and the occlusion expert paints a grey halo around

TABLE XI
TASK 2 CLASSIFIER TEST METRICS.

Class	Precision	Recall	F1	Support
Clean	0.993	0.995	0.994	3669
Salt-and-pepper	1.000	1.000	1.000	11007
Gaussian blur	0.999	1.000	0.999	11007
Occlusion	1.000	0.998	0.999	11007
Macro / overall acc.	0.998	0.998	0.998	acc 0.999

TABLE XII

TASK 2 HARD ROUTING: ORACLE VS. PREDICTED ROUTING ON THE TEST SET. 42 ENTRIES (0.1%) WERE MISROUTED; THE MEAN PSNR LOSS ON THOSE WAS 25.63 dB.

Condition	PSNR in	PSNR oracle	PSNR pred.	SSIM oracle	SSIM pred.
Clean	100.00	100.00	99.66	1.000	0.999
Salt-and-pepper	16.38	24.63	24.63	0.778	0.778
Gaussian blur	27.78	24.36	24.37	0.764	0.764
Occlusion	13.61	21.00	21.01	0.690	0.690

a black background (9.7 dB). So the cost of a routing error depends on which expert receives the image, and it is highest when a clean image goes to the occlusion expert.

The second finding is that the specialists do not beat the universal autoencoder. With oracle routing they average 23.33 dB against 23.63 dB for Task 1 (salt 24.63 versus 24.76, blur 24.36 versus 24.84, occlusion 21.00 versus 21.30). The specialists share one architecture chosen by a short search and each sees only a third of the corruption distribution, while the universal model trains for 60 epochs on everything. I did not run the ablation that would separate these causes. Like the universal model, the blur specialist ends below its input PSNR (24.36 versus 27.78 dB). The specialist curves (Fig. 18) agree with this. The salt-and-pepper and blur specialists end at nearly the same validation objective (about 0.26), the occlusion specialist stays much higher (about 0.37), and all three were still falling slowly at epoch 40, so longer training would probably have helped them.

C. Task 3: soft mixture-of-experts

Analysis. The examples in Fig. 19 show how the gate behaves. Clean images get $w \approx [0.97\text{--}0.99, 0, 0, 0.01\text{--}0.03]$ and a PSNR of about 55 dB, so they pass through almost untouched. Salt-and-pepper images send 0.88–1.00 of the weight to the salt expert, and a high-severity blur image sends 0.97 to the blur expert. The occlusion examples are different: the occlusion expert gets only 0.64–0.79 and the identity branch keeps 0.21–0.35, so part of the black rectangle is copied through, and the error maps show the largest errors inside the rectangles (19.7 dB at high severity). The validation curve in Fig. 20 falls from 0.206 to about 0.184 over the 32 epochs, a gain of roughly 11%, and it is flat in the last ten epochs, so the fine-tuning helped but did not change the picture dramatically. The training loss is lower only because it is a weighted loss that is smaller than L1+(1-SSIM) by construction.

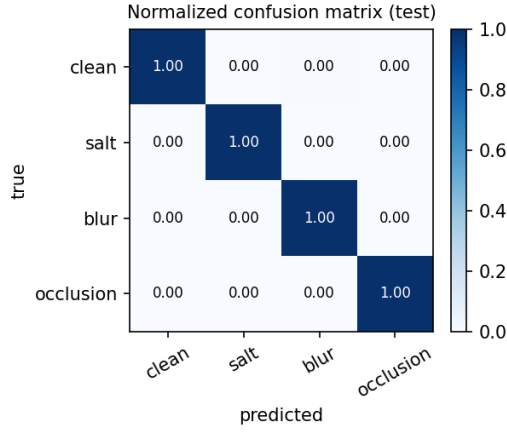


Fig. 15. Normalised confusion matrix of the corruption classifier on the test manifest.

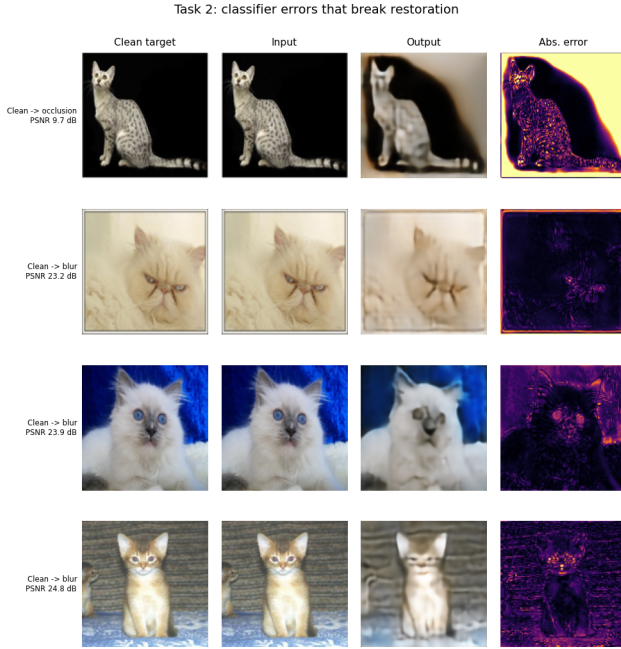


Fig. 16. Cases where a classifier error breaks the restoration.

On single corruptions the soft MoE is the best system (Table XVII): 25.18, 25.62 and 21.84 dB for salt-and-pepper, blur and occlusion, against 24.76, 24.84 and 21.30 for Task 1 and 24.63, 24.36 and 21.00 for oracle hard routing. The mean is 24.22 dB against 23.63 and 23.33. This comparison is not perfectly fair. The MoE starts from the Task 2 weights and then trains for another 32 epochs, so it has seen more training than the other systems, and part of its gain may come from that.

On the mixed stress test the order reverses (Table XVI). The universal autoencoder reaches 19.42 dB, the soft MoE 19.07 dB and hard routing 18.88 dB. Soft blending beats a hard choice (+0.19 dB), but the gate was trained with one label per image, so it cannot pick two experts for two simultaneous

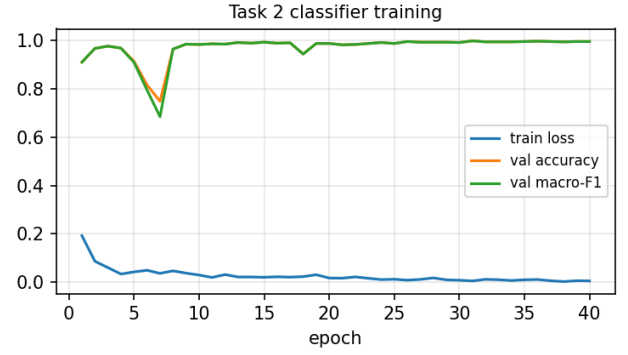


Fig. 17. Classifier training curves.

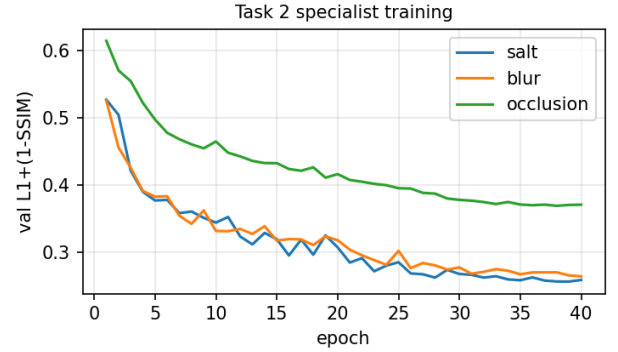
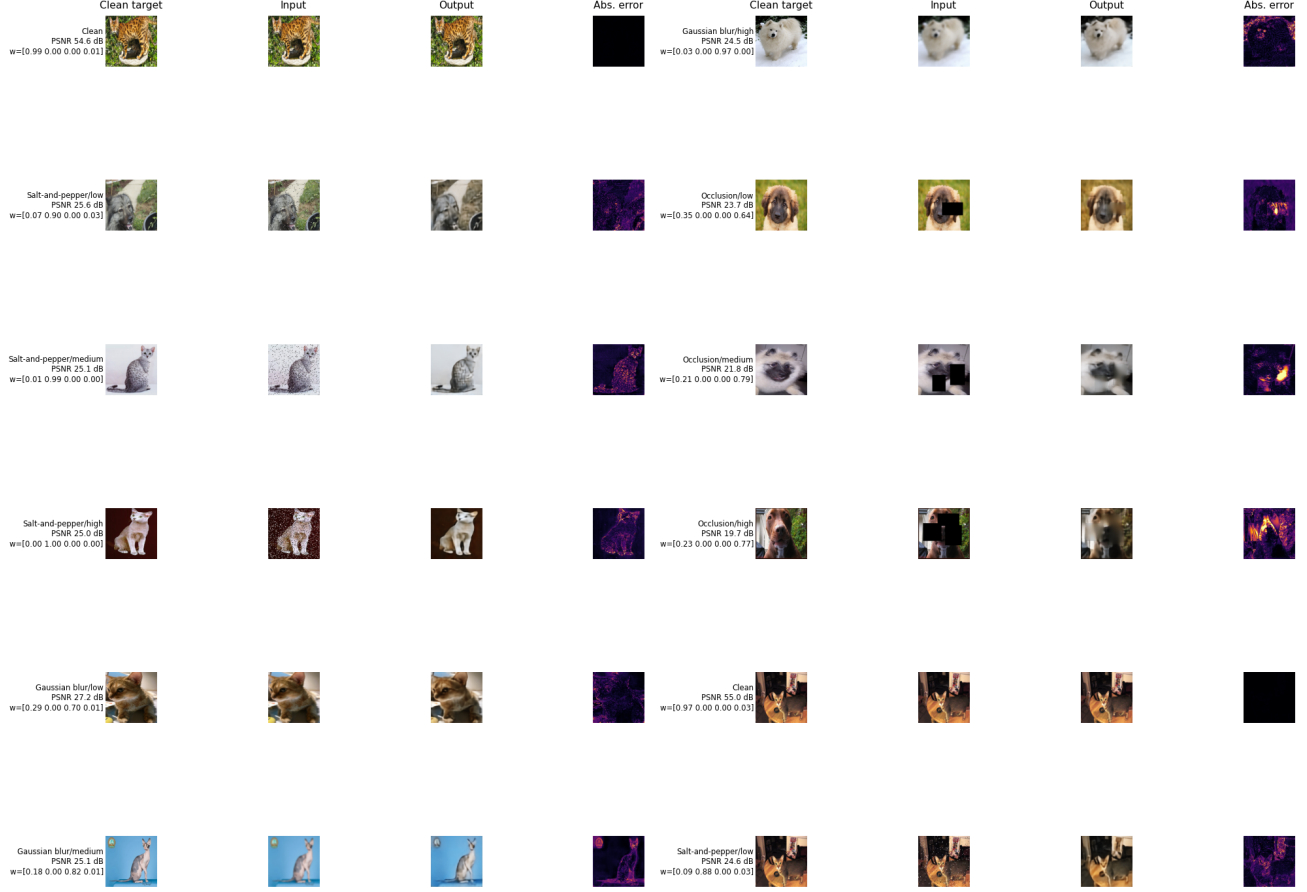


Fig. 18. Validation objective $L1+(1-SSIM)$ of the three specialists during training.

corruptions, and the experts have never seen the second corruption. The three systems are within 0.55 dB of each other, so routing gives no clear advantage here. Fig. 24 shows why. In the blur plus salt-and-pepper examples the gate sends 0.98–0.99 to the salt expert and ignores the blur, so the dots are removed but the picture stays blurry. In the occlusion plus blur examples it sends 0.88–0.91 to the blur expert and none to the occlusion expert, so the black rectangles stay black and light up as the brightest regions of the error map. The gate picks one corruption and cannot select two experts, which is the limitation I expected from training on single labels.

On clean images the soft model scores 53.64 dB instead of the 100 dB of the hard identity bypass, because the identity branch gets a weight of about 0.95 and the experts leak the remaining 5% (Table XIV). No branch is inactive or collapsed (Table XV): the mean weights are 0.25, 0.28, 0.25 and 0.22, and each expert gets almost no weight on average on the wrong corruption type (at most 0.014; Fig. 22 shows the same per image). The gate has also learned to blend by severity. The identity branch keeps a share of 0.37–0.38 for low-severity blur and occlusion and only 0.05–0.21 at high severity, while the expert weight rises with severity (blur 0.62, 0.89, 0.94; salt-and-pepper 0.87, 0.95, 0.97). Fig. 21 and Fig. 22 summarise this routing. The box plots show that for clean and salt-and-pepper inputs the right branch gets a weight with a very

Task 3: soft mixture-of-experts (12 test examples)

Fig. 19. Task 3: twelve test examples. The vector w printed on the left is the gate weight for [identity, salt-and-pepper, blur, occlusion].TABLE XIII
TASK 3 SOFT MIXTURE-OF-EXPERTS TEST RESULTS.

Condition	PSNR in	PSNR out	SSIM out	L1 out
Clean	100.00	53.64	0.999	0.0017
Salt-and-pepper	16.38	25.18	0.803	0.0408
Gaussian blur	27.78	25.62	0.807	0.0386
Occlusion	13.61	21.84	0.777	0.0506

narrow spread, while for blur and occlusion the spread is wide: the identity branch has a median of about 0.1 for blur and 0.27 for occlusion, with some images up to 0.6. Fig. 23 contrasts a case where one expert dominates (a medium salt-and-pepper image with $w = [0, 1.00, 0, 0]$) with a low-severity blur image where the gate hedges between three branches ($w = [0.40, 0, 0.40, 0.19]$), which is what I would expect when the blur is barely visible. During joint training the batch-mean weights settled after about five epochs (Fig. 25). In that figure the identity branch averages about 0.37 because it is measured on class-balanced batches, while Table XV averages over the test manifest, where only one entry in ten is clean, which is why it shows 0.25 there. I did not evaluate the gate on the

TABLE XIV
AVERAGE GATE WEIGHTS PER TRUE CORRUPTION TYPE AND SEVERITY (TEST SET).

Condition	w_0 id	w_1 salt	w_2 blur	w_3 occl.
clean/-	0.95	0.00	0.01	0.03
salt/low	0.11	0.87	0.00	0.03
salt/medium	0.04	0.95	0.00	0.01
salt/high	0.02	0.97	0.00	0.01
blur/low	0.37	0.00	0.62	0.01
blur/medium	0.11	0.00	0.89	0.01
blur/high	0.05	0.00	0.94	0.00
occlusion/low	0.38	0.00	0.00	0.62
occlusion/medium	0.29	0.00	0.00	0.71
occlusion/high	0.21	0.00	0.00	0.79

test set before fine-tuning, so I cannot say how far fine-tuning moved it.

The failure cases (Fig. 26) are the same black-background cat as in Task 1 (9.2 and 10.4dB, with correct routing at $w_{occ} = 0.83$ and 0.74) and high-severity salt-and-pepper images with saturated colours. There the green lawn comes back olive and the pink background beige even though the

TABLE XV

ROUTING HEALTH: NO BRANCH IS INACTIVE WHEN ITS MEAN WEIGHT AND DOMINANCE FRACTION ARE NON-ZERO.

Branch	Mean weight	Frac. $w > 0.5$	Mean w on other types
identity	0.253	0.139	0.175
salt-expert	0.279	0.298	0.000
blur-expert	0.247	0.280	0.002
occlusion-expert	0.221	0.280	0.014

TABLE XVI

TWO SIMULTANEOUS CORRUPTIONS (BLUR+SALT-AND-PEPPER, OCCLUSION+BLUR).

System	PSNR	SSIM
no restoration	14.71	0.452
Task 1 universal AE	19.42	0.692
Task 2 hard routing (predicted)	18.88	0.666
Task 3 soft MoE	19.07	0.682

gate sends 0.87–0.98 of the weight to the right expert. So these failures are in the restoration and not in the routing.

D. Task 4: face-to-sketch

Analysis. The cGAN is clearly better than the baseline of using the grayscale photo as the sketch (PSNR 15.61 versus 6.73 dB, SSIM 0.538 versus 0.289, L1 0.106 versus 0.416). Results differ a lot by style. Style 2 is worst (12.34 dB, L1 0.153) and Style 3 is best (18.66 dB), but the test set has 619, 381 and only 46 images of the three styles, so the Style 3 number rests on very few samples. The Style 2 ground-truth sketches use dense dark hatching whose stroke positions cannot be predicted, so an L1-trained generator produces a smoother average and scores poorly even when the face is right.

The style condition is used by the model. With the correct style the L1 is 0.1056 and with a wrong style it is 0.1467, which is 39% higher. Fig. 27 shows three distinct looks from the same photo: light thin lines for Style 1, heavy dark shading for Style 2 and a softer grey rendering for Style 3.

Training was stable (Fig. 29). The discriminator losses stayed near 0.3 and the generator adversarial loss settled near 2.6 after about 20 epochs, above the equilibrium value of 0.69, so the discriminator stays ahead of the generator without overpowering it. Validation L1 fell from 0.118 to about 0.10 within 20 epochs and ended at 0.095 after 150 epochs, while the training reconstruction loss kept falling from 0.28 to 0.155 (Fig. 30). The training value is probably higher because it is measured with augmentation and dropout switched on. Fig. 31 shows the progress: after the first epochs the faces are smeared blobs, by the middle snapshots hair, eyes and mouth have lines, and in the last snapshots the three styles are clearly separate.

The highest-L1 failures (Fig. 28) are all women with long dark or voluminous hair. The ground-truth hair is drawn with many strokes that the generator renders as lighter, smoother hair. Photos with reflective sunglasses also give a garbled eye region (last row of Fig. 27).

TABLE XVII

PSNR OF THE FOUR RESTORATION SYSTEMS ON THE SAME TEST MANIFEST.

	PSNR (dB)	T1 universal	T2 oracle	T2 predicted	T3 soft MoE
Clean	24.94	24.94	100.00	99.66	53.64
Salt-and-pepper	24.76	24.76	24.63	24.63	25.18
Gaussian blur	24.84	24.84	24.36	24.37	25.62
Occlusion	21.30	21.00	21.01	21.01	21.84
All corrupted	23.63	23.33	23.33	23.33	24.22

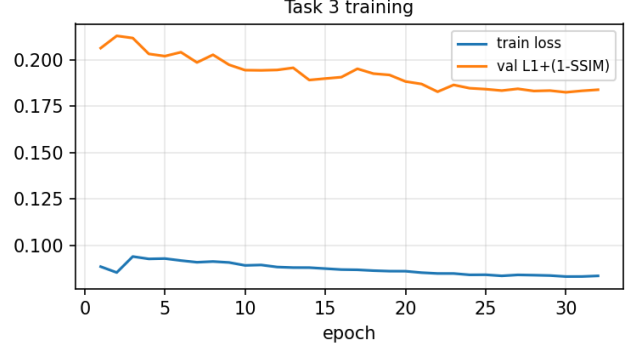


Fig. 20. Task 3 training and validation curves (joint stage).

IX. APPLICATION ARCHITECTURE

I made the interface design in Google Stitch from a written brief describing the page structure, colours, controls, cards and responsive layout (Fig. 33). I did this after the React and Tailwind CSS app was already working, so the design documents the interface and was not used as code. The Stitch screens show the same four workspaces as the app. The sample values on them (latency, PSNR, hardware name) are placeholder text generated by Stitch and are not results from this work. nginx serves the React bundle and proxies `/api` to a FastAPI container. The backend checks the file type and size, converts to RGB, resizes to 128×128 , applies the requested corruption with the same code used in training, runs the ONNX models with ONNX Runtime and returns the images, the routing information and the timing. Fig. 32 shows how the containers fit together and Fig. 34 shows the four workspaces in use, with the uploaded image, the corrupted input, the restored output and the routing information visible in each. The endpoints are `/api/health`, `/api/universal`, `/api/hard-route`, `/api/soft-moe` and `/api/face2sketch`. docker compose up --build starts everything at `http://localhost:8080`, with the ONNX files mounted from `./models`.

A. ONNX export

The universal autoencoder, the classifier, the three specialists, the full soft-MoE pipeline (gate, experts and temperature baked in) and the generator are exported with a dynamic batch axis. Table XIX gives the largest absolute difference between PyTorch and ONNX Runtime outputs on random inputs.

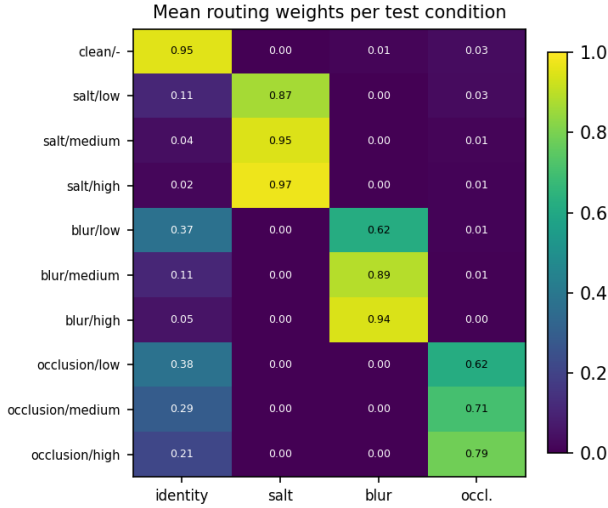


Fig. 21. Mean routing weights per test condition.

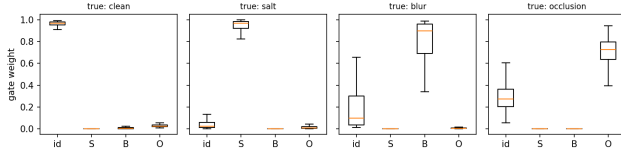


Fig. 22. Distribution of gate weights per true corruption type.

X. LIMITATIONS

Occlusion removes information, so every model can only paint plausible content there. Pixel metrics understate the visual quality and also penalise other valid answers. PSNR, SSIM and L1 also favour blurry outputs, and I did not compute perceptual metrics such as LPIPS or FID.

The GAN Optuna objective measures fidelity to the paired sketch and not realism, and short trials may rank settings differently from full training. FS2K is small (about 900 training pairs after the split), and the photos of the three styles come from different sources, so style may be tied to image content. The Task 4 test set is also imbalanced across styles (619/381/46).

The classifier and experts are tuned for the four specified corruptions. Unseen degradations such as JPEG, motion blur or Gaussian noise are outside the training distribution, and I did not test any. For Gaussian blur, every bottleneck restorer ends below the PSNR of the blurred input (24.4–24.8 versus 27.8dB) and SSIM also drops, so by pixel metrics these models are worse than doing nothing on blur. A deblurring network with skip connections or a residual design would be needed. The Task 2 comparison between specialists and the universal model is not an ablation because the training budgets differ. Finally, everything runs at 128×128 , and the app resizes larger uploads, which throws away detail.



Fig. 23. An example where one expert dominates (top) and one where weights are spread over several branches (bottom).

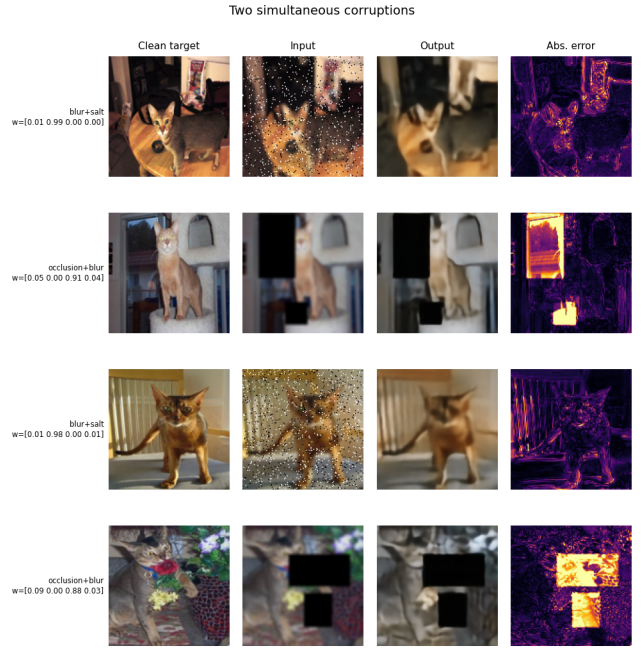


Fig. 24. Two simultaneous corruptions handled by the soft model.

XI. CONCLUSION

The soft mixture-of-experts was the best strategy on single corruptions (mean 24.22 dB against 23.63 for the universal autoencoder and 23.33 for hard routing), but the universal autoencoder was best when two corruptions were combined (19.42 dB). Routing therefore helps only when the input matches what the experts were trained for. Classifying the corruption was not the bottleneck (99.9% accuracy, oracle and predicted routing equal), and every bottleneck autoencoder failed on blur by ending below the PSNR of the blurred input. In the Optuna studies the loss weight α (autoencoders), regularisation (classifier), gate temperature and balance weight (MoE) and the two GAN learning rates mattered most, while network width and latent size mattered little. The style condition controls the cGAN output (L1 0.106 with the correct style against 0.147 with a wrong one), although fidelity is limited by the unpredictable strokes of Style 2 and the small Style 3 test

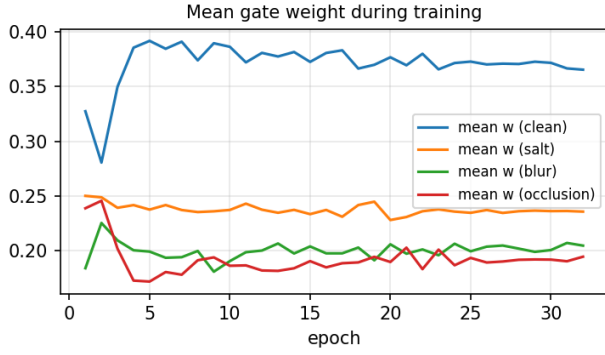


Fig. 25. Mean gate weights during warm-up (first epochs) and joint fine-tuning.

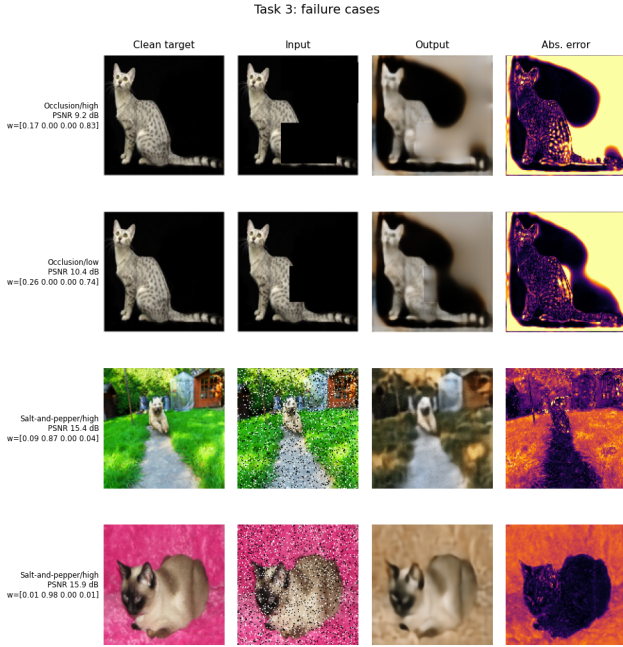


Fig. 26. Task 3 failure cases with the gate weights $[w_0, w_1, w_2, w_3]$ printed on the left.

set. With more time I would try a residual or skip-connection design for blur, train on mixed corruptions, run an ablation on specialist training budget, and add LPIPS for the GAN.

B *

Appendix B

Use of AI Tools I used Claude (Anthropic) throughout this project. It wrote the first version of the code (data pipeline, model definitions, Optuna and MLflow scripts, ONNX export, FastAPI backend, React frontend and Docker files) and drafted the text of this report from my results. I ran all training on Google Colab, found and fixed the Colab and repository setup problems (the notebook clone and FS2K unzip steps, Git LFS and the folder layout), ran the Docker deployment and took the screenshots. The numbers in this report come from my own runs and result files. The generated code was checked with

TABLE XVIII

TASK 4 TEST RESULTS AGAINST THE PAIRED GROUND-TRUTH SKETCH. STYLE-CONDITIONING CHECK: L1 WITH THE CORRECT STYLE 0.1056 VS. 0.1467 WITH A WRONG STYLE.

Set	PSNR	SSIM	L1	N
Style 1	17.39	0.582	0.0794	619
Style 2	12.34	0.450	0.1532	381
Style 3	18.66	0.666	0.0649	46
All styles (cGAN)	15.61	0.538	0.1056	1046
Grayscale-photo baseline	6.73	0.289	0.4155	1046

TABLE XIX

ONNX EXPORT CHECK: MAXIMUM ABSOLUTE DIFFERENCE BETWEEN PYTORCH AND ONNX RUNTIME OUTPUTS.

Model	File	Max abs. diff	MB	Match
universal_udae	universal_udae.onnx	5.1e-07	16.05	yes
classifier	classifier.onnx	9.3e-10	1.18	yes
specialist_salt	specialist_salt.onnx	3.0e-07	16.15	yes
specialist_blur	specialist_blur.onnx	4.5e-07	16.15	yes
specialist_occlusion	specialist_occlusion.onnx	3.6e-07	16.15	yes
soft_moe	soft_moe.onnx	4.2e-07	49.65	yes
generator	generator.onnx	2.1e-06	117.27	yes
generator_batch1		0.0e+00		yes

CPU smoke tests on synthetic data, PyTorch versus ONNX parity checks, API tests and a browser test. I also used Google Stitch to make the interface design in Fig. 33, after the React frontend already existed. No other AI tools were used.

D *

Appendix D

Reproducibility To reproduce the results, run `pip install -r requirements.txt`, then `bash scripts/run_all.sh`, then `docker compose up --build`. The README in the repository lists the dataset download steps, the expected folder layout, the model download links and the demo workflow.

E *

References

- [1] P. Vincent, H. Larochelle, Y. Bengio and P.-A. Manzagol, "Extracting and composing robust features with denoising autoencoders," in *Proc. ICML*, 2008.
- [2] K. Zhang, W. Zuo, Y. Chen, D. Meng and L. Zhang, "Beyond a Gaussian denoiser: Residual learning of deep CNN for image denoising," *IEEE Trans. Image Process.*, vol. 26, no. 7, 2017.
- [3] D. Pathak, P. Krähenbühl, J. Donahue, T. Darrell and A. Efros, "Context encoders: Feature learning by inpainting," in *Proc. CVPR*, 2016.
- [4] B. Li, X. Liu, P. Hu, Z. Wu, J. Lv and X. Peng, "All-in-one image restoration for unknown corruption," in *Proc. CVPR*, 2022.
- [5] Z. Wang, A. Bovik, H. Sheikh and E. Simoncelli, "Image quality assessment: From error visibility to structural similarity," *IEEE Trans. Image Process.*, vol. 13, no. 4, 2004.
- [6] R. Jacobs, M. Jordan, S. Nowlan and G. Hinton, "Adaptive mixtures of local experts," *Neural Computation*, vol. 3, no. 1, 1991.
- [7] N. Shazeer *et al.*, "Outrageously large neural networks: The sparsely-gated mixture-of-experts layer," in *Proc. ICLR*, 2017.
- [8] M. Mirza and S. Osindero, "Conditional generative adversarial nets," arXiv:1411.1784, 2014.
- [9] P. Isola, J.-Y. Zhu, T. Zhou and A. Efros, "Image-to-image translation with conditional adversarial networks," in *Proc. CVPR*, 2017.
- [10] O. Ronneberger, P. Fischer and T. Brox, "U-Net: Convolutional networks for biomedical image segmentation," in *Proc. MICCAI*, 2015.

- [11] E. Perez, F. Strub, H. de Vries, V. Dumoulin and A. Courville, "FiLM: Visual reasoning with a general conditioning layer," in *Proc. AAAI*, 2018.
- [12] D. Ulyanov, A. Vedaldi and V. Lempitsky, "Instance normalization: The missing ingredient for fast stylization," arXiv:1607.08022, 2016.
- [13] T. Akiba, S. Sano, T. Yanase, T. Ohta and M. Koyama, "Optuna: A next-generation hyperparameter optimization framework," in *Proc. KDD*, 2019.
- [14] O. Parkhi, A. Vedaldi, A. Zisserman and C. Jawahar, "Cats and dogs," in *Proc. CVPR*, 2012.
- [15] D.-P. Fan *et al.*, "Facial-sketch synthesis: A new challenge," *Machine Intelligence Research*, vol. 19, 2022.

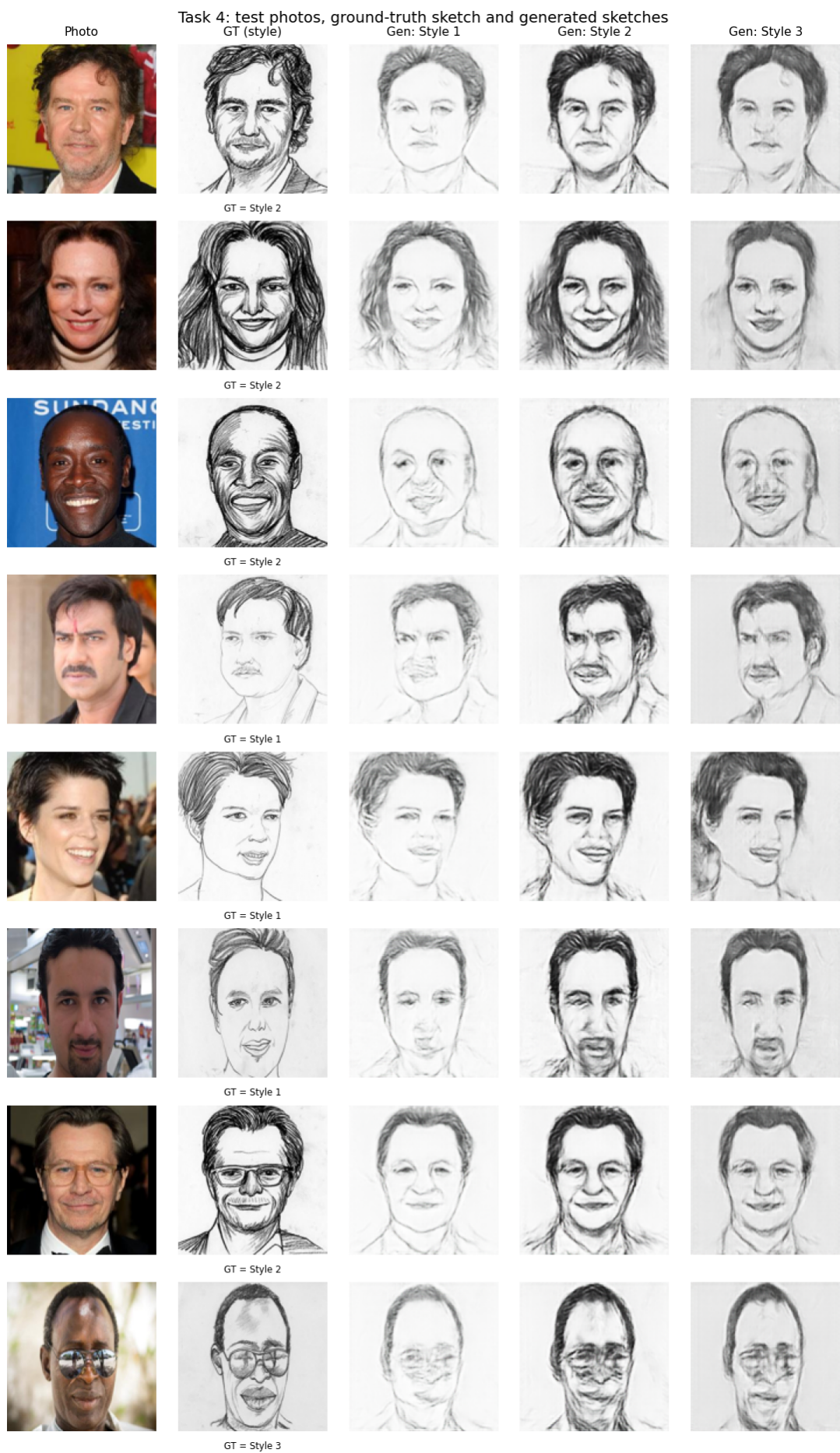


Fig. 27. Task 4: test photographs, ground-truth sketches and the three generated styles.

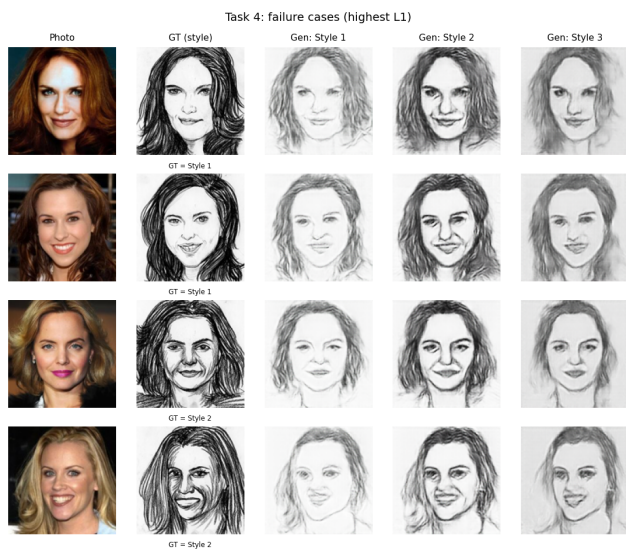


Fig. 28. Task 4 failure cases (highest L1).

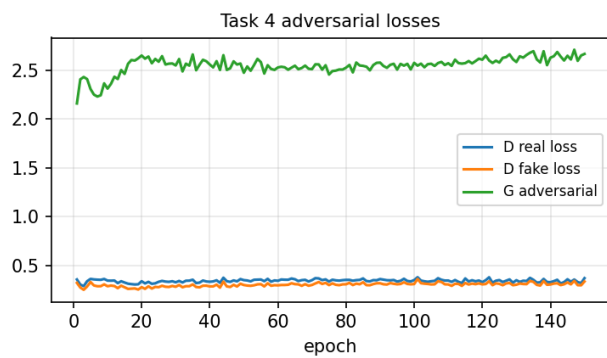


Fig. 29. Adversarial training losses.

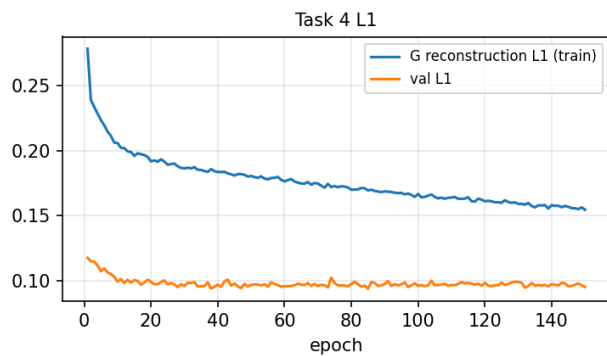


Fig. 30. Generator reconstruction loss and validation L1.



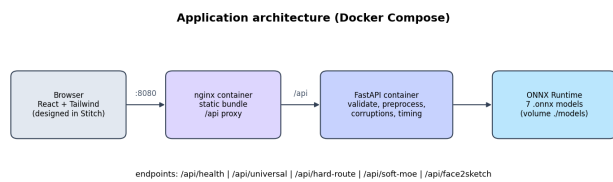


Fig. 32. Deployment architecture.

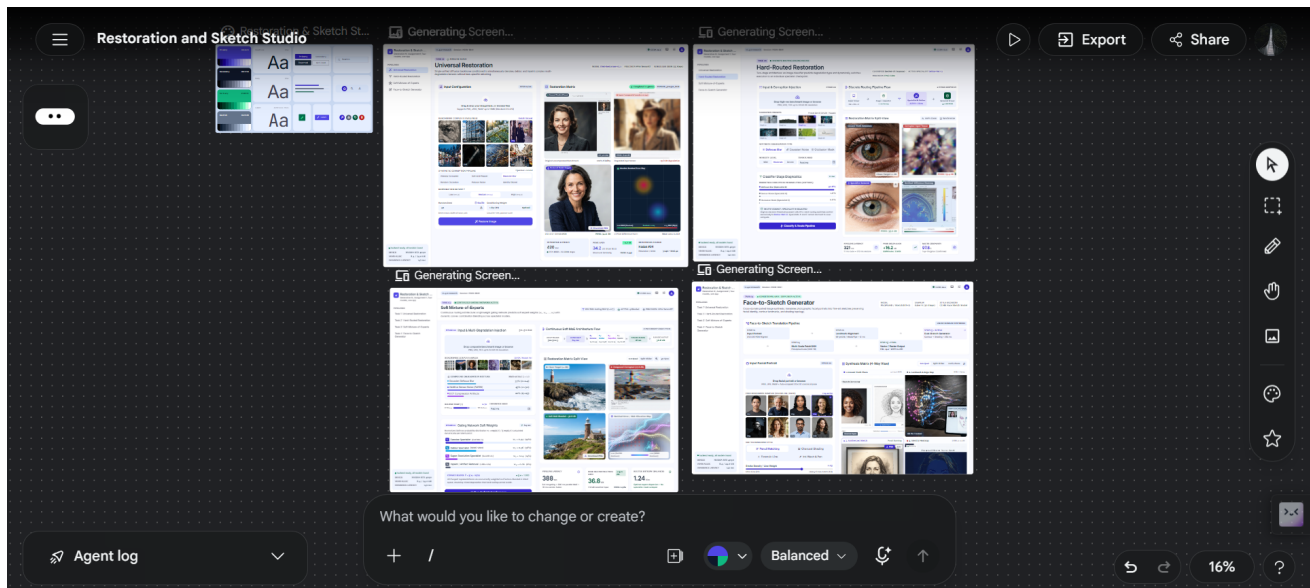


Fig. 33. Google Stitch design of the interface: design-system tokens and the four workspace screens (Universal Restoration, Hard-Routed Restoration, Soft Mixture-of-Experts, Face-to-Sketch Generator).

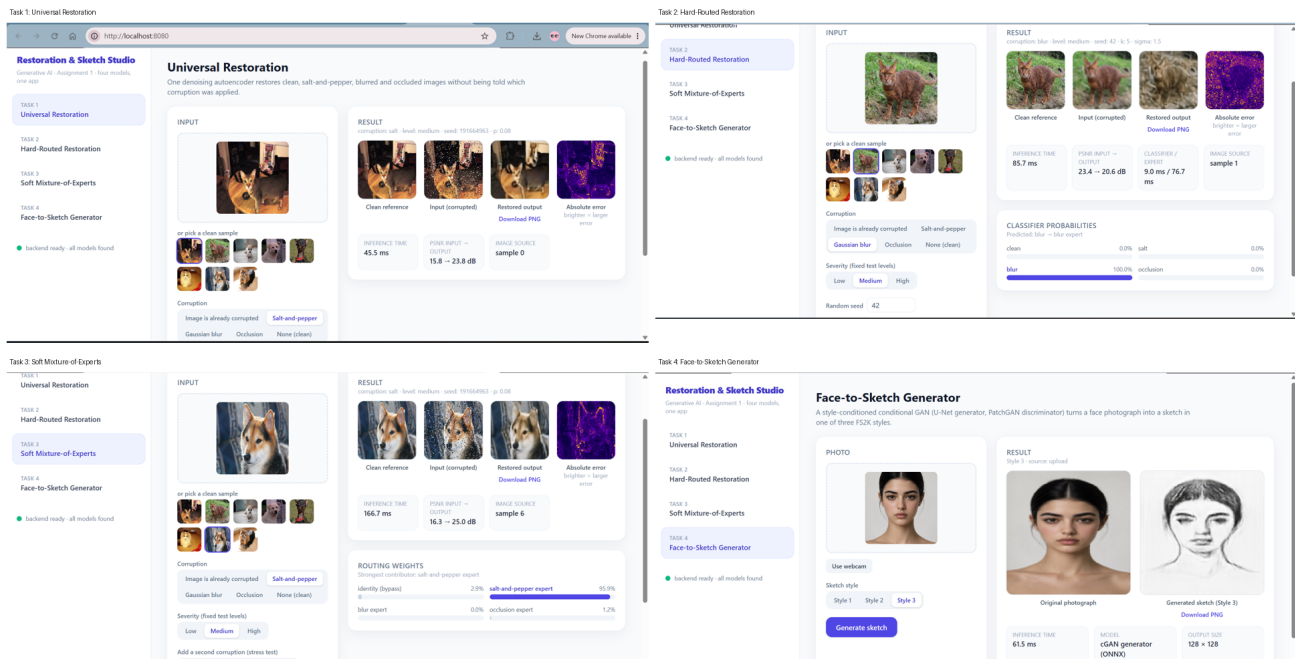


Fig. 34. Screenshots of the four workspaces: Universal Restoration, Hard-Routed Restoration, Soft Mixture-of-Experts Restoration and Face-to-Sketch Generator.